Tools and Techniques of Programming in R

Lesson-Three

Programming in R with the ITERATION Logic

Objective of the chapter: The objective of this chapter is to illustrate the idea of repeated execution of a set of code till the user desires. The logic structure used for this purpose is called a 'loop' structure. In the literature of systems, the structure is known as the 'Iteration logic' structure. This logic structure is inevitable in computer programming, because most often we require the repeated execution of a part or whole of a set of programming statements until we meet our requirement.
Pre-requisite: Knowledge of a programming language will be an added advantage.

4.1 The LOOP or ITERATION structure

we shall now study the most important logic structure used in programs. This logic structure enables us to perform repetitive a task .Such a logic structure is popularly known as **loop structure** also. This logic structure is used when we require executing the same set of codes for a desirable number of times. The desirable number may also be kept **indefinite** depending on the user's choice during the execution. There are two forms of iteration, namely, (1) **pre-test iteration** and (2) **post-test iteration**. In case of pre-test iteration, a predicate, commonly known as a condition or conditional expression, is checked first; if it evaluates to true, only then the set of statements under that structure is executed. In case of post-test iteration, a set of statements under the loop is executed first and then a conditional expression is checked to decide whether the repetition is to be done or not. Thus, the statements in the post-test iteration are executed at least once. On the contrary, the statements under the pre-test iteration may not be executed even once.

The following figure depicts the flowchart forms of both types of iterations:

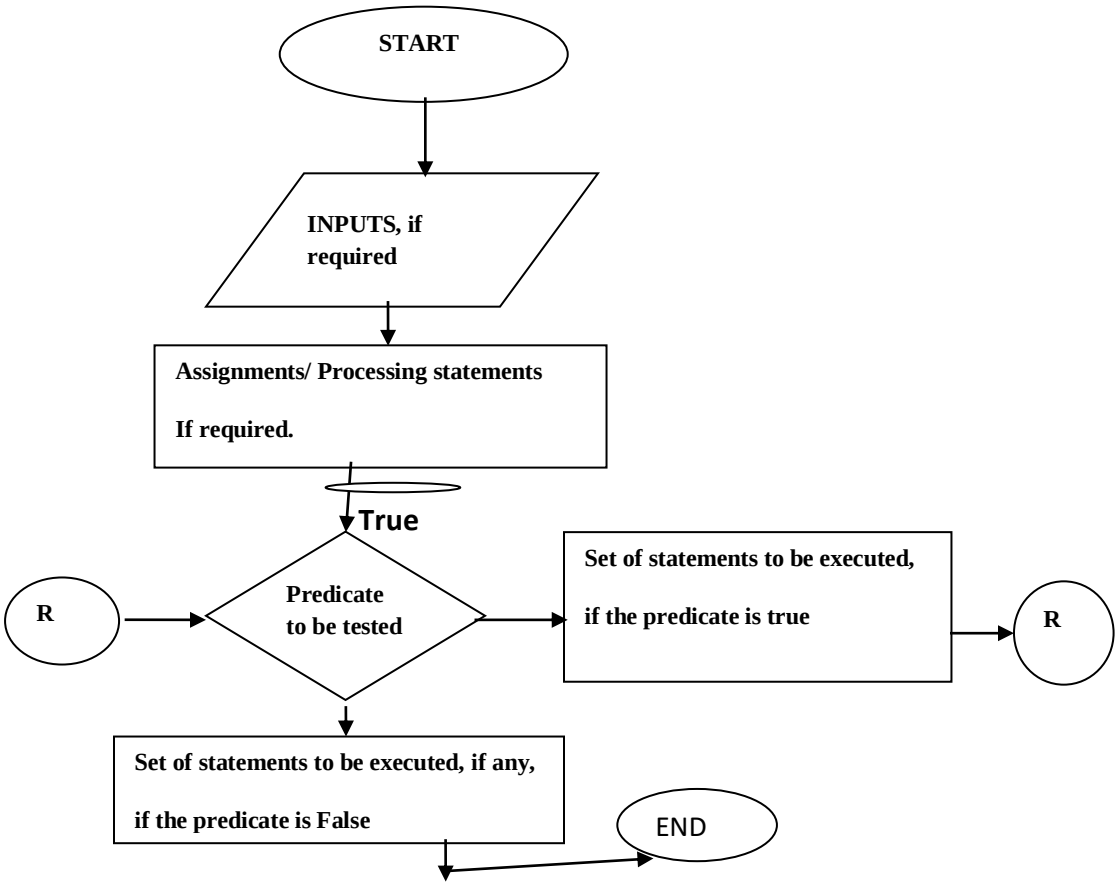

FIG 4.1 : THE PRE-TEST ITERATION

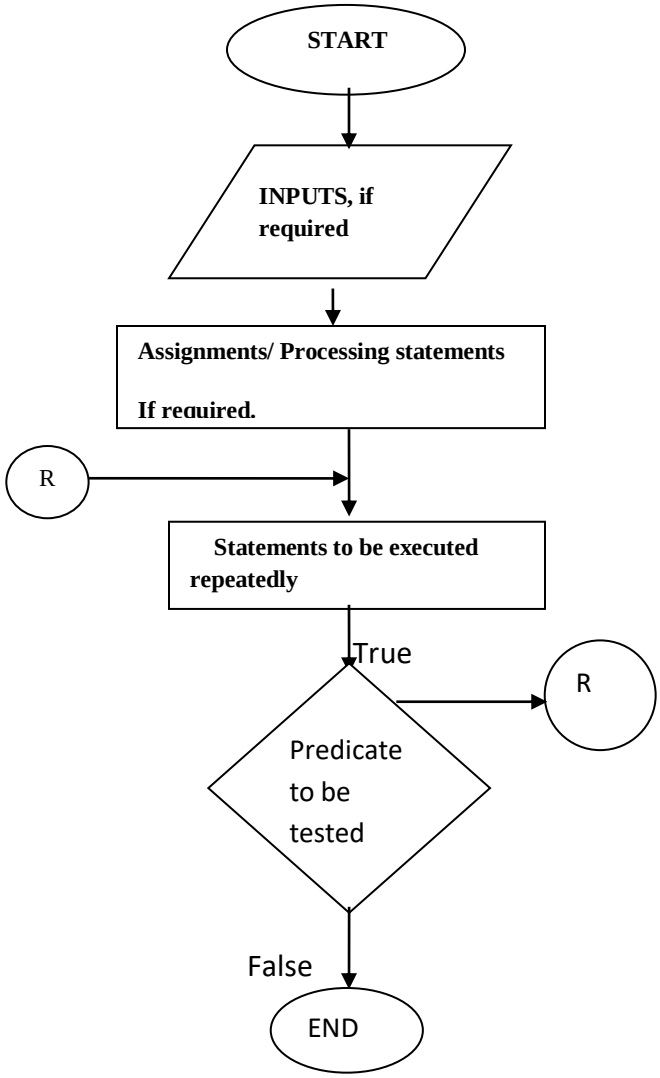

Figure: 4.2: THE Post-TEST Iteration.

4.2 THE ITERATIONS STRUCTURES SUPPORTED BY R.

Pre-Test Iteration. R supports two structures for pre-test iteration. One is, while(conditional-expression) {set of statements to be repeated } structure and the other is for (expression) {set of statements to be repeated structure} . The actual syntax of the loop structures are shown below:

The basic syntax for creating a ‘ while’ loop in R is as shown below:

```
while (conditional-expression to be tested) {
    Set of R statements to be executed
}
```

Let us have some illustrated loop with this structure.

Figure: 4.3a

```
Count=1
while(count<=5) {
 print(“ Welcome to R ‘while’ loop”)
  Count= Count+1
}
```

If this code is executed in R environment, then we shall have the following output:

Fig

```
[1] "Welcome to R 'while' loop"

[1] "Welcome to R 'while' loop"

[1] "Welcome to R 'while' loop"

[1] "Welcome to R 'while' loop"
```

Now, we shall study the structure of the R for loop. The basic syntax of this loop is as shown below:

for (variable in sequence) {
set of statements to be executed repeatedly
}

Here, *variable* refers to any R variable used for taking a set of values mentioned by the ***sequence*** one by one to execute the set of statements written for repeated execution.

Here, the increment of the loop variable will always be by 1.However, sometimes, we need to increase the loop variable by any desired step size. For this purpose, the loop structure should be written as follows:

for(loop-variable in seq(beginning-value, ending value, step-size)){ loop statements}

Another point to be borne in mind that R does not compare the initial value of the loop variable with the ending value. It first executes the loop with the initial value, increments or decrements the its value by the appropriate step value and then compares the resulting value with the ending value before going to execute the loop statements for the second time to ascertain whether to continue the loop or to discontinue. The code snippet illustrates this feature:

```
> for(i in seq(1,0,-2))print(i)
[1] 1
```

Now,let us have some illustration of execution of a for loop below:

```
s=0
for(I in 1:10){
s=s+I
}
print(paste("The sum of the first 10 natural numbers
is:  ",s))
```

Figure: 4.4a

Here, the sequence is the set of natural numbers from 1 to 10 and I is any R variable. The output of the above code is shown in the figure below:

```
> s=0
> for(I in 1:10){
+ s=s+I
+ }
> print(paste("The sum first 10 natural numbers is: ",s))
[1] "The sum first 10 natural numbers is:  55"
>
```

Figure: 4.4b

4.3 The 'repeat' loop structure

There is no loop structure in R strictly following the definition of post-test iteration. However, there is a third loop structure in R that may be used in that format to some extent depending on the way how the user utilizes the loop structure. This loop structure is known as ***repeat*** loop structure. Truly speaking, the ***repeat*** loop is a never ending loop i.e. an infinite loop. But, there is a control statement in R that can be used to come out of the loop at any point of action. It is the ***break*** statement. This statement is usually written under a conditional statement to break the repetition. If this conditional statement is written just before the ending brace of the loop, then only we get the flavor of the post-test iteration. In general, the loop structure can be demonstrated by using the flowcharting symbols as shown in the figure below:

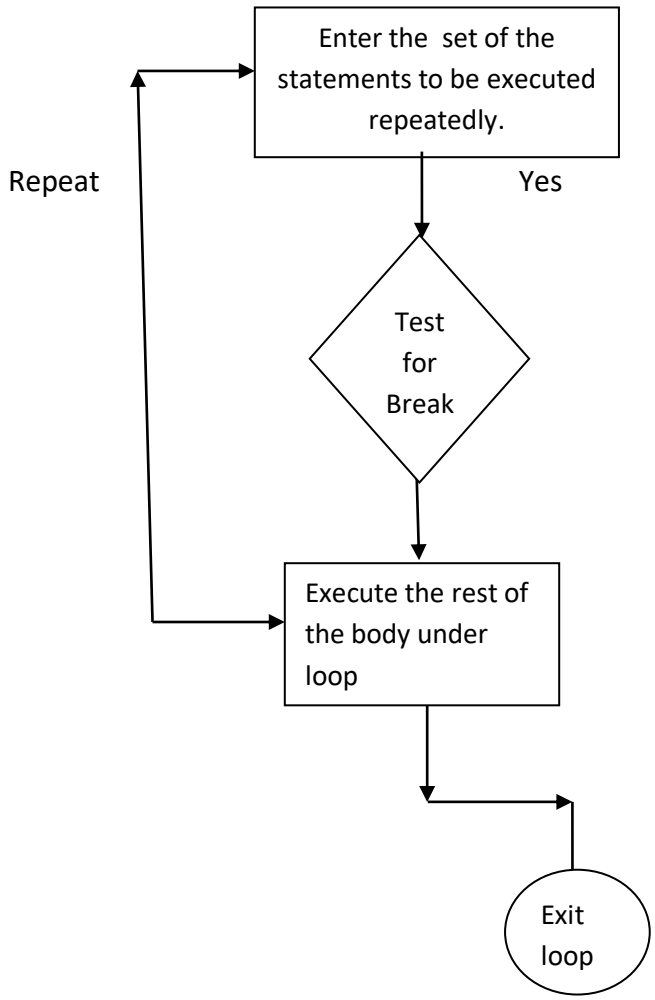

Figure: 4.5.The repeat loop structure.

In terms R, the structure can be written as follows:

repeat { set of statements to be executed repeatedly}

There is another control statement in R that is often used in a ***for loop***. It is the keyword: ***next.*** When the R system encounters the ***next*** command, it suspends execution of the remaining statements in the body of the loop , goes to the beginning of the loop and starts execution of the loop statements with the next value of the loop variable, if any.

Let us have an illustration of the ***repeat loop*** structure as in the figure below:

```
count=1
> repeat{
+ print(" This is the repeat loop")
+ count=count+1
+ if (count>3)break
+ }
```

The output of this loop is shown in the figure below:

```
[1] " This is the repeat loop"
[1] " This is the repeat loop"
[1] " This is the repeat loop"
```

Figure: 4.6b

4.4 Illustrative examples using iteration

Let us now consider the solution of a few problems for which we shall have to make use of the loops.

Problem 4.1. It is required to obtain the sum of consecutive even numbers starting from 2 until the sum just exceeds 1000 and then to show the sum and the number of even numbers added.

Task Analysis. This is a very easy problem in which we shall not require any input value, because the numbers to added can be generated through calculation. The solution in R code is given below:

```
# Program to determine the sum of even numbers.
n=2
s=0
cnt=0
while(s<=1000){
s=s+n
cnt=cnt+1
n=n+2}
print(paste("The desired sum of consecutive even numbers is: ",s))
print(paste("The number of even numbers added is: ",cnt))
print("End of Program.....Thank You")
```

Figure: 4.7a. Sum of Consecutive even numbers

The displays on the R console is shown in the figure below when the above code is run:

```
> # Program to determine the sum of even numbers.
> n=2
> s=0
> cnt=0
> while(s<=1000){
+ s=s+n
+ cnt=cnt+1
+ n=n+2}
> print(paste("The desired sum of consecutive even numbers is: ",s))
[1] "The desired sum of consecutive even numbers is:  1056"
> print(paste("The number of even numbers added is: ",cnt))
[1] "The number of even numbers added is:  32"
> print("End of Program.....Thank You")
[1] "End of Program.....Thank You"
```

Problem 4.2. It is required to obtain the reversed form of a given number by using some R script.

Task Analysis: To obtain the reversed form of a given number, we proceed as explained below:We obtain the rightmost digit of the number by dividing it by 10 and collecting the remainder. This digit can then be stored in a variable and then the given number can be replaced with its integer quotient by treating it as the given number and updating the variable by using the formula: variable=variable with the old value*10 + new remainder. The process is to be repeated until the given number is reduced to zero. However, the process described above will not show the zeroes at the end of the given number, if any. So, we have added some additional codes in R to concatenate zeroes at the leading positions as may be desired. These codes are underlined for the readers' perusal. We have used paste() function for the concatenation. The keyword *sep="",implies no spaces in the concatenated zeroes .*

```
# Reversing a given number
cat("Enter the any whole number")
n=scan(n=1)
rn=0
cnt=0
while(n>0){
r=n%%10
if(r==0)
cnt=cnt+1
rn=rn*10+r
n=n%/%10 # This is to obtain the integer part of the quotient.
}
if(cnt>0)
rn=as.character(rn)
while(cnt>0){
rn=paste('0',rn,sep="",collapse=NULL)
cnt=cnt-1
}
print(paste("The reversed number is: ",rn))

print("End of program.... Thank you")
```

and collapse=NULL means the that the number is to be shown as a single string. This will give us the reversed number in the variable considered. The process is illustrated in the figure below:

Figure: 4.8a

```
# Reversing a given number
cat("Enter the any whole number")
n=scan(n=1)
rn=0
cnt=0
while(n>0){
r=n%%10
if(r==0)
cnt=cnt+1
rn=rn*10+r
n=n%/%10 # This is to obtain the integer part of the quotient.
}
if(cnt>0)
rn=as.character(rn)
while(cnt>0){
rn=paste('0',rn,sep="",collapse=NULL)
cnt=cnt-1
}
print(paste("The reversed number is: ",rn))
```

The displays on the R console are shown in the figure below:

Figure 4.8b

```
n=scan(n=1)
1: 999997654321
Read 1 item
> rn=0
> cnt=0
> while(n>0){
+ r=n%%10
+ if(r==0)
+ cnt=cnt+1
+ rn=rn*10+r
+ n=n%/%10
+ }
> if(cnt>0)
+ rn=as.character(rn)
> while(cnt>0){
+ rn=paste('0',rn,collapse=NULL)
+ cnt=cnt-1
+ }
> print(paste("The reversed number is: ",rn))
[1] "The reversed number is:  123456799999"
> print("End of program.... Thank you")
[1] "End of program.... Thank you"
> n=scan(n=1)
1: 1100000
Read 1 item
> rn=0
> cnt=0
> while(n>0){
+ r=n%%10
+ if(r==0)
+ cnt=cnt+1
+ rn=rn*10+r
+ n=n%/%10
+ }
> if(cnt>0)
+ rn=as.character(rn)
> while(cnt>0){
+ rn=paste('0',rn,sep="",collapse=NULL)
+ cnt=cnt-1}
> print(paste("The reversed number is: ",rn))
[1] "The reversed number is:  0000011"
> print("End of program.... Thank you")
[1] "End of program.... Thank you"
```

Problem 4. 3. A program in R is required to be developed to show the divisors of a given number.

Task Analysis: This is also a very easy task. A number is said to be divisible by another number if the division of the former number by the latter number leaves zero remainder. We shall have to think of the solution in the way a child will think of it. We shall start checking the divisibility starting from one (1) and continue the checking until we reach the half of the number, because, after the half of the number, only the number itself can be the divisor. So, we should not check beyond the half value to avoid unnecessary calculation.

The required codes in R have been shown in the figure below:

```
# Program to determine the divisors of a given number
cat("Enter any whole number")
n=scan(n=1)
i=1
print(paste("The divisorsof the number:  ",n,"are shown below:"))
for(i in 1:as.integer(n/2)){
if(n%%i==0)
print(i)
}
print(n)
print("END od the program....Thank YOU!")
```

When the above code is run, the following will be displays on the console:

```
> cat("Enter any whole number")
Enter any whole number> n=scan(n=1)
1: 28
Read 1 item
> i=1
> print(paste("The divisorsof the number:  ",n,"are shown below:"))
[1] "The divisorsof the number:  28 are shown below:"
> for(i in 1:as.integer(n/2)){
+ if(n%%i==0)
+ print(i)
+ }
[1] 1
[1] 2
[1] 4
[1] 7
[1] 14
> print(n)
[1] 28
> print("END od the program....Thank YOU!")
[1] "END od the program....Thank YOU!"
>
```

Figure: 4.9b

Problem 4.4. A number is called a *Perfect Number* if the sum of its divisors except itself equals the number. It is required to develop an R script for testing a given number as to whether it is a perfect number or not.

Task Analysis. We have already seen how to find out the divisors of a given number. So, the solution of this problem becomes a very easy one in the light of that concept. The only additional task we shall be required to do is obtaining the sum of the divisors and then to compare the sum with the given number to draw the required conclusion.

The requisite R script is shown in the figure below:

```
#Testing for a perfect number
cat("Enter a whole number to be tested")
N=scan(n=1)
sum=0
for(i in 1:as.numeric(N/2)){
if(N%%i==0)
sum=sum+i}
if(N==sum){
print("The number is a PERFECT NUMBER")
 } else {
print("It is not a Perfect Number")
  }
```

```
> # Testing for a perfect number
> cat("Enter a whole number to be tested")
Enter a whole number to be tested> N=scan(n=1)
1: 9998
Read 1 item
> sum=0
> for(i in 1:as.numeric(N/2)){
+ if(N%%i==0)
+ sum=sum+i
+ }
> if(N==sum){
+ print("The number is a PERFECT NUMBER")
+  } else {
+ print("It is not a Perfect Number")
+   }
[1] "It is not a Perfect Number"
> # Testing for a perfect number
> cat("Enter a whole number to be tested")
Enter a whole number to be tested> N=scan(n=1)
1: 496
Read 1 item
> sum=0
> for(i in 1:as.numeric(N/2)){
+ if(N%%i==0)
+ sum=sum+i
+ }
> if(N==sum){
+ print("The number is a PERFECT NUMBER")
+  } else {
+ print("It is not a Perfect Number")
+   }

[1] "The number is a PERFECT NUMBER"
```

Problem 4.5. Narcissistic Number. if an n-digit number is equal to the sum of the n th powers of its digits, it is called an n-narcissistic number. It is also sometimes known as an Armstrong number. For example;153, 1634, 558834 are such numbers.

It is required to develop an R script for checking whether a given number is a Narcissistic number or not.

Task Analysis: Here the first task is to find out the number of digits in the number. Next, we separate the digits from the number one by one, take the exponential value of the digit with the power equal to the number of digits in the number and sum up the values. At the end, we compare the sum of the exponential values of the digits with the given number and conclude.

The following figure contains the desired R script:

Here, we have used the paste() function of R for **string concatenation**. In the function, ***sep*** stands for separator and the **collapse=NULL** implies that the concatenated value will have to be stored as a single value without unnecessary spaces.

The screen displays in the R environment, when the script is run, is given in figure below:

```
# Testing for Narcissistic property of a given number
cat("Enter any number for the testing")
N=scan(n=1)
M=N # We save here the given number for later reference
d=0
while(N>0){
d=d+1
N=N%/%10 # Integer division
}
string=""
tot=0
N=M
t=d
while(N>0){
r=N%%10
tot=tot+r^d
# The following 5 lines of code may be omitted by a  novice programmer. It is to show: digits raised to #the desired
power sum up to give the given number
t=t-1
if(t!=0){
string=paste(string,paste(r,"^",d,"+  "),sep="",collapse=NULL) # This is for concatenation
} else {
string=paste(string,paste(r,"^",d),sep="",collapse=NULL) # This is also for concatenation without the # plus sign
for the last digit
}
N=N%/%10
}
if(M==tot){
print("It is a Narcissistic Number")
print(paste("Because",M,"=",string))
} else {
print(" It is not a Narcissistic Number")
}
print("Thank You")
```

```
> cat("Enter any number for the testing")
Enter any number for the testing> N=scan(n=1)
1: 153
Read 1 item
> M=N # We save here the given number for later reference
> d=0
> while(N>0){ d=d+1
+ N=N%/%10 # Integer division}
> string=""
> tot=0
> N=M
> t=d
> while(N>0){ r=N%%10
+ tot=tot+r^d
+ t=t-1
+ if(t!=0){
+ string=paste(string,paste(r,"^",d,"+  "),sep="",collapse=NULL)
+ } else { string=paste(string,paste(r,"^",d),sep="",collapse=NULL)}
+ N=N%/%10}
> if(M==tot){  print("It is a Narcissistic Number")
+ print(paste("Because",M,"=",string))
+ } else { print(" It is not a Narcissistic Number")
+ }
[1] "It is a Narcissistic Number"
[1] "Because 153 = 3 ^ 3 +  5 ^ 3 +  1 ^ 3"
> N=scan(n=1)
1: 1634
Read 1 item
> M=N # We save here the given number for later reference
> d=0
> while(N>0){
+ d=d+1
+ N=N%/%10 # Integer division}
> string=""
> tot=0
> N=M
> t=d
> while(N>0){
+ r=N%%10
+ tot=tot+r^d
+ t=t-1
+ if(t!=0){  string=paste(string,paste(r,"^",d,"+  "),sep="",collapse=NULL)
+ } else { string=paste(string,paste(r,"^",d),sep="",collapse=NULL)}
+ N=N%/%10}
> if(M==tot){ print("It is a Narcissistic Number")
> M=N # We save here the given number for later reference
> d=+ print(paste("Because",M,"=",string))
+ } else { + print(" It is not a Narcissistic Number")}
 [1] "It is a Narcissistic Number
[1] "Because 1634 = 4 ^ 4 +  3 ^ 4 +  6 ^ 4 +  1 ^ 4"
```

Figure:4.12b

Problem4. 6. A sequence of numbers is said to follow the Fibonacci series, if the first three terms are 0, 1, 1 and the subsequent terms of the series can be obtained by taking the sum of the preceding two terms. It is required to develop an R script to generate any desired number of terms of the series.

Task Analysis: The finding out of the terms of the Fibonacci series is a very common programming task. We usually perform the task by initializing two variables with the first two terms which are known to us and then take the sum of their values to store in a third variable to have the third term. Now, after printing the value of the first term, we store the value of the second term in the variable containing the first term and that of the third term into the variable containing the second term. This will put the value of the second term in the first variable. So, we go on printing the value of the first variable until the desired number of terms has been printed.

The required R script has been shown in the figure below:

```
# Program to print the numbers of Fibonacci series
cat(" How many terms of the Fibonacci series would you like to see?")
n=scan(n=1)
t1=0
t2=1
print(" The Fibonacci terms are as shown below:")
for(i in 1:n){
```

Figure: 4.13a

```
# Program to print the numbers of Fibonacci series
> cat(" How many terms of the Fibonacci series would you like to see?")
 How many terms of the Fibonacci series would you like to see?> n=scan(n=1)
1: 10
Read 1 item
> t1=0
> t2=1
> print(" The Fibonacci terms are as shown below:")
[1] " The Fibonacci terms are as shown below:"
> for(i in 1:n){
+ print(t1)
+ t=t1+t2
+ t1=t2
+ t2=t
+ }
[1] 0
[1] 1
[1] 1
[1] 2
[1] 3
[1] 5
[1] 8
[1] 13
[1] 21
[1] 34
> print(" End the program.... Thank You")
[1] " End the program.... Thank You"
```

Figure: 4.13b

Problem 4.7. It is required to test whether a given number is a prime number or not.

Task Analysis: We know that a whole number is said to be a prime number if it has no other divisor or factor except itself and unity (1). As 1 has no other factor except itself, it is excluded from the list of prime numbers. So, we are required to test from 2 to the given number to determine whether it has any other factor except 1 and itself. As 1 and the number itself are the common factors of all numbers, we need not test for them. Also, we know that if a number has any factor at all, then at least one of the factors must lie within the square root of the number. So, we can confine our checking from 2 to the integer part of the positive square root of the given number. If we do not get any factor or divisor within that range, we can safely conclude that the given number is a prime number. R provides us with sqrt() function to find out the square root of any real number. We shall make use of the function.

The required R script is given in the figure below:

```
# Testing for primality of a given number
cat(" Enter the number to be tested for primality")
n=scan(n=1)
flag=0
sr=as.integer(sqrt(n))
i=2
while(i<=sr){
if(n%%i==0){
flag=1
break
}
i=i+1
}
if(flag==0){
print(" The given number is a prime number")
} else {
print(" The given number is not a prime number")
}
print(" End of program")
```

The screen displays in the R environment will be as shown in the figure below when the codes are run:

```
> # Testing for primality of a given number
> cat(" Enter the number to be tested for primality")
 Enter the number to be tested for primality> n=scan(n=1)
1: 2
Read 1 item
> flag=0
> sr=as.integer(sqrt(n))
> i=2
> while(i<=sr){
+ if(n%%i==0){ flag=1; break; } i=i+1+ }
> if(flag==0){
+ print(" The given number is a prime number")
+ } else {
+ print(" The given number is not a prime number")
+ }
[1] " The given number is a prime number"
> print(" End of program")
[1] " End of program"
> cat(" Enter the number to be tested for primality")
 Enter the number to be tested for primality> n=scan(n=1)
1: 23
Read 1 item
> flag=0
> sr=as.integer(sqrt(n))
> i=2
> while(i<=sr){ if(n%%i==0){ flag=1;  break }
+ i=i+1 }
> if(flag==0){ print(" The given number is a prime number")
+ } else { print(" The given number is not a prime number") }
[1] " The given number is a prime number"
> print(" End of program")
[1] " End of program"
> cat(" Enter the number to be tested for primality")
 Enter the number to be tested for primality> n=scan(n=1)
1: 22
Read 1 item
> flag=0
> sr=as.integer(sqrt(n))
> i=2
> while(i<=sr){ if(n%%i==0){ flag=1; break+ }
+ i=i+1}
> if(flag==0){ print(" The given number is a prime number")
+ } else { print(" The given number is not a prime number")
```

Problem 4.8. It is required to develop an R script to determine the factorial value of a number.

Task Analysis: We know that the factorial value of a number is equal to the product of the integers from 1 to that number. So, the task is very easy and the R script for the task is shown in the figure below:

```
# Program to find out the factorial of a number
cat("Enter the number the factorial of which is required")
n=scan(n=1)
fact=1 # variable where we shall accumulate the factorial value
for(i in 1:n){
fact=fact*i
}
print(paste(" The factorial value of ", n,"is: ", fact))
print(" End of the Program.....Thank you!")
```

The following figure shows the screen displays in the R environment when the program is run:

```
> # Program to find out the factorial of a number
> cat("Enter the number the factorial of which is required")
Enter the number the factorial of which is required> n=scan(n=1)
1: 5
Read 1 item
> fact=1 # variable where we shall accumulate the factorial value
> for(i in 1:n){
+ fact=fact*i}
> print(paste(" The factorial value of ", n,"is: ", fact))
[1] " The factorial value of  5 is:  120"
> print(" End of the Program.....Thank you!")
[1] " End of the Program.....Thank you!"
```

Figure:4.15b

Nested loop.

A **nested loop** is nothing but a loop within a loop. In order to solve some problems, we need to constructed nested loops. It is somewhat like ***nested if*** statements. The nesting can be done up to any depth. However, too much of nesting makes the program logic a bit complicated.

Let us see some illustrative problems in which nested loops will be required.

Problem 4. 9. Let it be required to find out all the perfect numbers within 10000.

Task Analysis: The logic **of** finding out the perfect numbers has already been discussed earlier in this chapter. Now in order to find out all such perfect numbers within 10000,we need not accept any input, rather, we are required to check all numbers within 10000. Since, it is known that the first perfect number is 6, so, we start our checking all the natural numbers within 10000, starting from 6.

The required program is shown in the figure below:

```
# Program to find out all the perfect numbers within 10000
for(i in 6:10000){
tot=0
for(j in 1:as.integer(i/2)){
if(i%%j==0)
tot=tot+j}
if(i==tot)
```

It may be observed that the outer loop has been used to generate the numbers one by one automatically while the inner loop has been used to find out all the divisors of the number under consideration.

The following figure shows the displays in the R environment when the program is run.

Figure:4.16b

```
> # Program to find out all the perfect numbers within 10000
> for(i in 6:10000){
+ tot=0
+ for(j in 1:as.integer(i/2)){
+ if(i%%j==0)
+ tot=tot+j
+ }
+ if(i==tot)
+ print(i)
+ }
[1] 6
[1] 28
[1] 496
[1] 8128
```

Problem 4.10. Let us find out all the Armstrong numbers within 9999.

Task Analysis. We have already seen the logic of finding out **Armstrong numbers** or **Narcissistic Numbers** earlier in this chapter. In that example, we tested a given input. Now, we are required to find out all such numbers within 9999. So, here, we have to check all natural numbers from 1 to 9999. This checking will, therefore, require a nested loop. The outer loop is for generating numbers to be checked and the inner loop is for testing each of them.

The following figure shows the code in R:

Figure:4.17a

```
# Testing for Narcissistic property of a given number
cat("ALL the ARMSTONG numbers within 9999")
for(N in 1:9999){
M=N # We save here the given number for later reference
d=0
while(N>0){
d=d+1
N=N%/%10 # Integer division}}
string=""
tot=0
N=M
t=d
while(N>0){
r=N%%10
tot=tot+r^d
t=t-1
if(t!=0){
string=paste(string,paste(r,"^",d,"+  "),sep="",collapse=NULL)
} else {
string=paste(string,paste(r,"^",d),sep="",collapse=NULL)}
N=N%/%10}
if(M==tot){
print(M)
print(paste("Because",M,"=",string))
}
}
print("Thank You")
```

The screen displays of the execution of the above code in R environment are shown in the figure below:

```
> # Testing for Narcissistic property of a given number
> cat("ALL the ARMSTONG numbers within 9999")
ALL the ARMSTONG numbers within 9999> for(N in 1:9999){
+ M=N # We save here the given number for later reference
+ d=0
+ while(N>0){
+ d=d+1
+ N=N%/%10 # Integer division}
+ string=""
+ tot=0
+ N=M
+ t=d
+ while(N>0){
+ r=N%%10
+ tot=tot+r^d
+ t=t-1
+ if(t!=0){
+ string=paste(string,paste(r,"^",d,"+  "),sep="",collapse=NULL)
+ } else {
+ string=paste(string,paste(r,"^",d),sep="",collapse=NULL)}
+ N=N%/%10}
+ if(M==tot){
+ print(M)
+ print(paste("Because",M,"=",string))}}
 [1] 1
[1] "Because 1 = 1 ^ 1"
[1] 2
[1] "Because 2 = 2 ^ 1"
[1] 3
[1] "Because 3 = 3 ^ 1"
[1] 4
[1] "Because 4 = 4 ^ 1"
[1] 5
[1] "Because 5 = 5 ^ 1"
[1] 6
[1] "Because 6 = 6 ^ 1"
[1] 7
[1] "Because 7 = 7 ^ 1"
[1] 8
[1] "Because 8 = 8 ^ 1"
[1] 9
[1] "Because 9 = 9 ^ 1"
[1] 153
[1] "Because 153 = 3 ^ 3 +  5 ^ 3 +  1 ^ 3"
[1] 370
[1] "Because 370 = 0 ^ 3 +  7 ^ 3 +  3 ^ 3"
[1] 371
[1] "Because 371 = 1 ^ 3 +  7 ^ 3 +  3 ^ 3"
[1] 407
[1] "Because 407 = 7 ^ 3 +  0 ^ 3 +  4 ^ 3"
[1] 1634
[1] "Because 1634 = 4 ^ 4 +  3 ^ 4 +  6 ^ 4 +  1 ^ 4"
[1] 8208
[1] "Because 8208 = 8 ^ 4 +  0 ^ 4 +  2 ^ 4 +  8 ^ 4"
[1] 9474
[1] "Because 9474 = 4 ^ 4 +  7 ^ 4 +  4 ^ 4 +  9 ^ 4"
> print("Thank You")
[1] "Thank You"
```

Figure:4.17b

Problem 4.11. Let us now develop an R script for finding out all the prime numbers within 1000.

Task Analysis. Testing a given number for its primality has already been discussed and illustrated in an earlier problem of this chapter. In that problem, we tested a given number. Here, we are required to check all the numbers within 1000. So, inputting is not necessary here; rather, we shall take all the natural numbers from 2 to 1000 automatically through a loop to check for the primality. Hence, we require a nested loop for the checking by generating natural numbers from 2 to 1000..

The following figure shows requisite program:

```
# Testing for primality of a given number
for(n in 2:1000){
flag=0
sr=as.integer(sqrt(n))
i=2
while(i<=sr){
if(n%%i==0){
flag=1
break
}
i=i+1
}
if(flag==0){
print(n)
}
```

Figure: 418b

```
> # Testing for primality of a given number
> for(n in 2:1000){
+ flag=0
+ sr=as.integer(sqrt(n))
+ i=2
+ while(i<=sr){
+ if(n%%i==0){
+ flag=1
+ break
+ }
+ i=i+1
+ }
+ if(flag==0){
+ print(n)
+ }
+ }
[1] 2
[1] 3
[1] 5
[1] 7
[1] 11
[1] 13
[1] 17
[1] 19
[1] 23
[1] 29
[1] 31
[1] 37
[1] 41
[1] 43
[1] 47
[1] 53
[1] 59
[1] 61
[1] 67
[1] 71
[1] 73
[1] 79
[1] 83
[1] 89
[1] 97
[1] 101
[1] 103
[1] 107
[1] 109
[1] 113
[1] 127
[1] 131
.....[1] 997
> print(" End of program")
[1] " End of program"
```

Problem 12. Some three-digit numbers show the property that the sum of the factorials of the digits equals the numbers. For example; 145=1!+4!+5!

Let us develop a program in R to find out all such numbers within 9999999.

```
# Program to determine the numbers the sum of the factorials of the digits of which equal the number.
for(n in 1:9999999)
{s=0
m=n
while(m>0){
r=m%%10
fact=1
while(r>1)
{fact=fact*r
r=r-1}
s=s+fact
m=m%/%10
}
if(s==n)
print(n)
}
```

Figure:4.19

Problem, 4.13.Some two-digit numbers have the property that the sum of the squares of the numbers equals the sum of the squares of the numbers with reversed digits. For xample:

$48^2 + 52^2 + 63^2 = 84^2 + 25^2 + 36^2$

Let us develop a program to find out all such two-digit triplets.

Task Analysis: Here, we shall have to check all possible triplets of two-digit numbers .The following figure shows the coding in R.

```
> # Program to determine the numbers the sum of the factorials of the digits of
which equal the number.
> for(n in 1:9999999)
+ {s=0
+ m=n
+ while(m>0){
+ r=m%%10
+ fact=1
+ while(r>1)
+ {fact=fact*r
+ r=r-1}
+ s=s+fact
+ m=m%/%10
+ }
+ if(s==n)
+ print(n)
+ }
[1] 1
[1] 2
[1] 145
[1] 40585
```

```
#Sum of squares of numbers equaling the sum of squares of the numbers formed with reversed digits
for(i in 10:99){
for(j in (i+1):98){
  for(k in (j+1):97)
{if(j>98||k>97)
next
     n1=i^2
      n2=j^2
      n3=k^2
      r1=i%%10
      r2=i%/%10
      rn1=r1*10+r2
      r1=j%%10
      r2=j%/%10
      rn2=r1*10+r2
      r1=k%%10
      r2=k%/%10
      rn3=r1*10+r2
      rn11=rn1^2
      rn22=rn2^2
      rn33=rn3^2
      str1=""
      str2=""
if((n1+n2+n3)==(rn11+rn22+rn33)){

str1=paste(str1,as.character(i),"^2+",as.character(j),"^2+",as.character(k),"^2",sep="",collapse=N
ULL)
  str2=
paste(str2,as.character(rn1),"^2+",as.character(rn2),"^2+",as.character(rn3),"^2",sep="",collapse=
NULL)
  print(paste(str1,"=",str2))
   }
   }
}
}
```

Fig.4.20a

The figures 4.20b and 4.20c show Partial displays on the R screen:

```
> #Sum of squares of numbers equaling the sum of squares of the numbers formed with reversed digits
> for(i in 10:99){
+ for(j in (i+1):98){
+   for(k in (j+1):97)
+ {if(j>98||k>97)
+ next
+      n1=i^2
+       n2=j^2
+       n3=k^2
+       r1=i%%10
+       r2=i%/%10
+       rn1=r1*10+r2
+       r1=j%%10
+       r2=j%/%10
+       rn2=r1*10+r2
+       r1=k%%10
+       r2=k%/%10
+       rn3=r1*10+r2
+       rn11=rn1^2
+       rn22=rn2^2
+       rn33=rn3^2
+       str1=""
+       str2=""
+ if((n1+n2+n3)==(rn11+rn22+rn33)){
+   str1=paste(str1,as.character(i),"^2+",as.character(j),"^2+",as.character(k),"^2",sep="",collapse=NULL)
+   str2= paste(str2,as.character(rn1),"^2+",as.character(rn2),"^2+",as.character(rn3),"^2",sep="",collapse=NULL)
+   print(paste(str1,"=",str2))
+    }
+    }
+ }
+ }
[1] "10^2+13^2+43^2 = 1^2+31^2+34^2"
[1] "10^2+20^2+23^2 = 1^2+2^2+32^2"
[1] "10^2+24^2+65^2 = 1^2+42^2+56^2"
[1] "10^2+25^2+64^2 = 1^2+52^2+46^2"
[1] "10^2+31^2+45^2 = 1^2+13^2+54^2"
[1] "10^2+35^2+41^2 = 1^2+53^2+14^2"
[1] "10^2+35^2+87^2 = 1^2+53^2+78^2"
[1] "10^2+37^2+85^2 = 1^2+73^2+58^2"
[1] "10^2+40^2+89^2 = 1^2+4^2+98^2"
[1] "10^2+42^2+67^2 = 1^2+24^2+76^2"
[1] "10^2+47^2+62^2 = 1^2+74^2+26^2"
[1] "10^2+47^2+97^2 = 1^2+74^2+79^2"
[1] "10^2+49^2+80^2 = 1^2+94^2+8^2"
[1] "10^2+53^2+89^2 = 1^2+35^2+98^2"
[1] "10^2+59^2+83^2 = 1^2+95^2+38^2"
[1] "10^2+63^2+68^2 = 1^2+36^2+86^2"
```

$$x+\frac{x^2}{2}+\frac{x^3}{3}+\ldots\ldots\text{up to n terms.}$$

And so on.

It is required to develop an R-script to obtain sum of the series for sets of values of x and n.

Task Analysis: The task is very easy for a novice programmer also because the power sign is supported by R. But, we should not go on evaluating such exponential expressions; because for very large n, the computation will have a high time complexity. We may rather use an intermediate variable, say t, for evaluating different exponential forms of x through a simple multiplication of the value of x in the preceding term by another x. To make further clear, we show below the computational part for evaluation of different exponential forms of x.

```
Input x,n
 Let t=1; sum=0
   For each I from 1 to n Do
   t=t*x
   sum=sum+t/i
   Endfor
```

The following figure shows the R code for the purpose:

```
# Sum of Series
repeat{
cat(" Enter the value of x followed by number of terms to be added")
v=scan(n=2)
x=v[1]
n=v[2]
sum=0
t=1
for(i in 1:n){
t=t*x
sum=sum+t/i
}
print(paste(" The sum of the series for x=",x,"and n=",n,"is:",sum))
ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'")
if(ch=="y"||ch=="Y"){
print(" Please carry on")
} else {
print("Thank you for running the program")
break
}
}
```

Fig:4.21b

```
> # Sum of Series
> repeat{
+ cat(" Enter the value of x followed by number of terms to be added")
+ v=scan(n=2)
+ x=v[1]
+ n=v[2]
+ sum=0
+ t=1
+ for(i in 1:n){
+ t=t*x
+ sum=sum+t/i}
+ print(paste(" The sum of the series for x=",x,"and n=",n,"is:",sum))
+ ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y) for
'No'")
+ if(ch=="y"||ch=="Y"){
+ print(" Please carry on")
+ } else {
+ break}}
Enter the value of x followed by number of terms to be added1: 2.5
2: 1234
Read 2 items
[1] " The sum of the series for x= 2.5 and n= 1234 is: Inf"
Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'n
> # Sum of Series
> repeat{
+ cat(" Enter the value of x followed by number of terms to be added")
+ v=scan(n=2)
+ x=v[1]
+ if(ch=="y"||ch=="Y"){
+ print(" Please carry on")
+ } else {
+ print("Thank you for running the program")
+ break}
+ }+ n=v[2]
+ sum=0
+ t=1
+ for(i in 1:n){
+ t=t*x
+ sum=sum+t/i}
+ print(paste(" The sum of the series for x=",x,"and n=",n,"is:",sum))
+ ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y) for
'No'")
 Enter the value of x followed by number of terms to be added1: 2.5
2: 50
Read 2 items
[1] " The sum of the series for x= 2.5 and n= 50 is: 2666351073503914496"
Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'n
[1] "Thank you for running the program"
```

Problem 4.15. It is required to develop an R script to determine the sum of the following series for different sets of values of x and n:

$x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^5}{5!} \pm$ **……. Up to n terms**

Task Analysis: Here also, we shall follow the logic similar to the above solution. This we shall require not only the exponential expressions in x, but also constant coefficients in factorial form. Further, there is a change of sign in alternate terms. The basic part of the algorithm may be as shown below:

1. **Input x,n**
2. **Sum=0**
3. **Term=-1**
4. **For each i from 1 to n do**
5. **Term=-Term*x/i**
6. **Sum=Sum+Term**
7. **Endfor**

It may be noted that the statement for the evaluation of Term determines each term with appropriate sign and factorial value. No additional calculation is needed for the determination of the factorials. This is, therefore, the optimum way of determining the sum of the series.
The R script for evaluating such a series is shown in the figure below:

```
# series2
repeat{
cat("Enter the value of x followed by the value of number of terms n")
v=scan(n=2)
x=v[1]
n=v[2]
term=-1
sum=0
for(i in 1:n){
term=-term*x/i
sum=sum+term
}
print(paste(" The sum of the series for x=",x,"and n=",n,"is:",sum))
ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'")
 if(ch=="y"||ch=="Y"){
 print(" Please carry on")
} else {
 print("Thank you for running the program")
 break
 }
 }
```

Figure:4.22a

The following figure shows the screen displays when the above code is run:

```
# series2
> repeat{
+ cat("Enter the value of x followed by the value of number of terms n")
+ v=scan(n=2)
+ x=v[1]
+ n=v[2]
+ term=-1
+ sum=0
+ for(i in 1:n){
+ term=-term*x/i
+ sum=sum+term
+ }
+ print(paste(" The sum of the series for x=",x,"and n=",n,"is:",sum))
+ ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'")
+  if(ch=="y"||ch=="Y"){
+  print(" Please carry on")
+ } else {
+  print("Thank you for running the program")
+  break
+  }
+  }
Enter the value of x followed by the value of number of terms n1: 2
2: 2
Read 2 items
[1] " The sum of the series for x= 2 and n= 2 is: 0"
Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'y
[1] " Please carry on"
Enter the value of x followed by the value of number of terms n1: 2.5
2: 5
Read 2 items
[1] " The sum of the series for x= 2.5 and n= 5 is: 1.16536458333333"
Would you like to continue?(Y/y) for 'yes' or (Y/y) for 'No'n
[1] "Thank you for running the program"
```

Problem 4.16. Consider the problem 3.4 of chapter three. We would like to modify output requirement slightly. The contractor demands to know total amount paid and the total number of workers paid at the of all payments.

Task Analysis: Here, we would require setting up an indefinite loop so that we can repeat the process any number of times. When we signal the end of input, the system would show the totals. The following figure shows the modified code:

```
# This is to calculate the wage of the workers under a contractor as per stated rules.
tot=0
cnt=0
repeat{
cat("Enter the hours worked in a week")
h=scan(n=1);
if (h>80){
print("Invalid Hours")
w=0
} else if(h<=35){
w=h*19
} else if(h<=60){
w=35*19+(h-35)*29
} else {
w=35*19+25*29+(h-60)*37
}
print(paste("The wage for this worker will be Rs. :",w,"for ",h,"hours of work"))
tot=tot+w
cnt=cnt+1
ch=readline(prompt="continue?(y/n)")
if(ch=="n"||ch=="N"){
break
}

}

print(paste("The total wage paid is Rs.: ",tot, "and the total number of workers paid is: ", cnt))

print("End of R script.... Thank You")
```

Figure:4.23a

Problem 4.17. It is required to develop a program to obtain GCD(HCF) and LCM of two given numbers.

Task analysis: Here, we adopt the Euclid's basic formula for the evaluation by using the Division-Remainder method. A number is said to be a GCD of another number if it can divide the latter without leaving any remainder; otherwise, we find out the greatest number that can divide both the numbers without leaving any remainder. The procedure that we follow here is that we divide the first number with the second one and obtain the remainder; if the remainder is zero, we stop immediately and conclude that the last divisor is the GCD of both; otherwise, we store the last divisor in the place of the last dividend and then we store the last remainder in the place of the divisor and then check the remainder again; we continue the process until we get a number that leaves zero remainder. The divisor that leaves zero remainder is the desired GCD. This method has been shown in the figure below through recursion.

To obtain the LCM of the numbers, we utilize the formula:

LCM X HCF of two numbers= Product of the numbers.

The following figure shows the required R script for the purpose:

```
# Finding out the HCF and LCM of two numbers
cat("Enter any two numbers of your choice")
m=scan(n=2)
a=as.integer(m[1]);b=as.integer(m[2])
p=a*b
repeat{
r=a%%b;a=b;b=r
if(r==0)break}
l=p/a;print(l)
print(paste("The HCF of the numbers is: ",a))
print(paste("And the LCM is: ",l))
print("End of program....")
```

The screen shots in the R environment have been in the figure below:

```
# Finding out the HCF and LCM of two numbers
> cat("Enter any two numbers of your choice")
Enter any two numbers of your choice> m=scan(n=2)
1: 5678
2: 7937
Read 2 items
> a=as.integer(m[1]);b=as.integer(m[2])
> p=a*b
> repeat{
+ r=a%%b;a=b;b=r
+ if(r==0)break}
> l=p/a;print(l)
[1] 45066286
> print(paste("The HCF of the numbers is: ",a))
[1] "The HCF of the numbers is:  1"
> print(paste("And the LCM is: ",l))
[1] "And the LCM is:  45066286"
> print("End of program....")
[1] "End of program...."
```

Figure:4.18a

Problem 4.18. A number is said to be a triad number if the double and triple of the number contain all separate digits with no repetition of any of them. It is required to print out all triad numbers within 10000.

Task Analysis: Here our checking should start from 1 and to be continued up to 10000. For each of the digits of the doubled number, we are required to compare with each of the digits of the tripled number, If no repetition is found among the digits, we conclude that the number under consideration is a triad number. The following figure shows the required code for the testing and printing of the triad numbers.

The following figure shows the partial screen displays when the program is run.

```
# Checking for  triad numbers within 10000
for (i in 1:10000){
d=2*i
t=3*i
while(d>0){
x=d%%10
d=d%/%10
flag=0
dt=t
while(dt>0){
y=dt%%10
if(x==y){
flag=1
break
}
dt=dt%/%10
}
if(flag==1)
break

}
if(flag==0){
print(paste(i,"is a triad Number"))
ch=readline(prompt="Press any key to continue")
}
}
```

Figure 4.18b

```
[1] "1 is a triad Number"
Press any key to continue
[1] "2 is a triad Number"
Press any key to continue
[1] "3 is a triad Number"
Press any key to continue
[1] "4 is a triad Number"
Press any key to continue
[1] "8 is a triad Number"
Press any key to continue

[1] "9 is a triad Number"

Press any key to continue

[1] "11 is a triad Number"

Press any key to continue

[1] "12 is a triad Number"

Press any key to continue

[1] "13 is a triad Number"

Press any key to continue

[1] "15 is a triad Number"

Press any key to continue

[1] "16 is a triad Number"

Press any key to continue

[1] "17 is a triad Number"

Press any key to continue

[1] "18 is a triad Number"

Press any key to continue

[1] "19 is a triad Number"

Press any key to continue
```

Chapter Summary: ***Here, we have learnt different types of loop structures that R supports***
R supports three loop structures, namely, 'while loop',' for loop' and 'repeat loop'. The purpose of all types of loops is the same; they differ in their syntax structures and the nature of execution. We have also studied a large number of illustrative examples implementing each loop structure. To explore the arena of 'looping', every reader should go through the solutions of a large range of problems from very simple to very hard, before stepping forward to solve the problems in the exercises.

EXERCISES FOUR

TypeI: MCQ style

1. Which of the following code-set skips the first 50 iterations ?
A. for(i in 1:100) { if(i<=50)next}}
B. for(i in 1:100) {if(i<=49){next}}
C.for(i in 1:100) {if(i<=51){next}}
D. All of the mentioned
E. None of the mentioned

2. Which of the following code snippet stops looping after 10 iterations ?
A.
```
for(i in 1:100) {
    print(i)
        if(i > 10) {
         break
    }
}
```
B.
```
for(i in 1:100) {
    print(i)
        if(i > 9) {
         break
    }
}
```
C.
```
for(i in 1:100) {
    print(i)
        if(i < 10) {
         break
    }
}
```
D. All of the mentioned

3.What is printed for the following code: ***for(i in 1:5){print(i);i=i+2}***
A. [1] 1
[1] 2
[1] 3

[1] 4
[1] 5
B. [1] 1
[1] 3
[1] 5
C. None of the mentioned
D. Either of the mentioned.
4. What is printed when the following code is executed at the R prompt: ***for(i in seq(3,10,3))print(i);print(i)***
A. [1] 3
[1] 6
[1] 9
[1] 11
B. [1] 3
[1] 6
[1] 9
[1] 9
C. None of the mentioned
D. Either of the mentioned
5. What is printed when the following code is executed at the R prompt? ***for(i in 1:6){if (i%%2==0) next else print(i)};print("Bottom of the Loop")***
A. Error: unexpected symbol in "for(i in 1:6){if (i%%2==0) next else print(i)}print"
B. [1] 1
[1] 3
[1] 5
C. [1] 1
[1] 3
[1] 5
[1] "Bottom of the Loop"
D. None of the mentioned points.
6. What is printed when the following code is executed at the R prompt*? for(i in 1:6){if (i%%2) next else print(i)};print("Bottom of the Loop")*
A. [1] 2
[1] 4
[1] 6
[1] "Bottom of the Loop"
B. [1] 1
[1] 3
[1] 5
[1] "Bottom of the Loop"
C. [1] 1
[1] 2
[1] 3

[1] 4
[1] 5
[1] 6
[1] "Bottom of the Loop"
D. None of the mentioned.
7. What is printed when the following code is executed at the R prompt*? for(i in 1:6)if (i%%2)print(i)else break;print("Bottom of the LOOP")*
A. print("Bottom of the LOOP")
Error: unexpected input in "print("'"
B. [1] 1
[1] "Bottom of the LOOP"
C. None of the mentioned
8. What is printed when the following code is executed at the R prompt*? i=0;while(i<5){if(i<2){i=i+2;next}else {i=i+2;print(i)};print("Bottom of the LOOP")}*
A. Error: unexpected symbol in "while(i<5){if(i<2){i=i+2;next}else {i=i+2;print(i)}print"
B. [1] 4
[1] "Bottom of the LOOP"
[1] 6
[1] "Bottom of the LOOP"
C. None of the mentioned
9. What is printed when the following code is executed at the R prompt*? i=0;repeat{if(i<3){i=i+2;print(i)}else{i=i+1;break};print("Bottom of the LOOP")}*
A. [1] 2
[1] "Bottom of the LOOP"
[1] 4
[1] "Bottom of the LOOP"
B. Error: unexpected ')' in:
"i=0;repeat{if(i<3){i=i+2;print(i)}else{i=i+1;break};print("BottoM")}
)"
C. None of the mentioned
10.* What is printed when the following code is executed at the R prompt*? for(i in 0:4)for(j in 0:4)for(k in 0:4)if(i+j+k==2) print(paste("i=",i,"j=",j,"K=",k))
A. It will show nothing
B. [1] "i= 0 j= 0 K= 2"
[1] "i= 0 j= 1 K= 1"
[1] "i= 0 j= 2 K= 0"
[1] "i= 1 j= 0 K= 1"
[1] "i= 1 j= 1 K= 0"
[1] "i= 2 j= 0 K= 0"
C. It will generate an error message
D. None of the mentioned
<u>Type II: Develop R scripts for the following problems:</u>

1. **To obtain the sum of the first n natural numbers for some given n without using any formula.**
2. **To obtain all the whole numbers which are divisible by some given p but not by some other given q.**
3. **To generate a table of any given number n.**
4. **To obtain all the groups of three successive numbers within 1000 that have the property that the square of the middle one is greater by unity than the product of the other two numbers. For example, $18^2 = 17*19+1$**
5. **To find out the series of five consecutive numbers within 1000,for which the sum of the squares of the first three is equal to the sum of the squares of the last two. For example, $(-2)^2 + (-1)^2 + 0^2 = 1^2 + 2^2$**
6. **To find out Pythagorean numbers x, y, z within 100 such that $z^2 = x^2 + y^2$.**
7. **To print the value of a given amount in words.**
8. **To obtain the sum of the following series for some given values of x and n: (i) $x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \ldots\ldots$ up to n terms.**

   (ii) $\frac{x^2}{2} - \frac{x^4}{4} + \frac{x^6}{6} - \ldots\ldots\ldots$ uo to n terms

   (iii) $\frac{x^1}{1!} - \frac{x^3}{3!} + \frac{x^5}{5!} - + \ldots\ldots\ldots$ up to n terms

   (iv) $\frac{x^2}{2!} - \frac{x^4}{4!} + \frac{x^6}{6!} - + \ldots\ldots\ldots$ up to n terms

   (i) $x - \frac{x^3}{2} + \frac{3*x^5}{2*4} - \frac{3*5*x^7}{2*4*6} + - \ldots\ldots$up to n terms
9. **Twin prime numbers are primes that differ by 2(e.g. 3 and 5, 101 and 103). To find out all twin primes within 10000.**
10. **To find out the list of months with a Friday the 13th for any given year.**

<u>Recommended books for further study</u>

1. **The art of programming through algorithms and flowcharts by Anil Bikas Chaudhuri, Fire Wall Media.**
2. **R in a nutshell by Joseph Adler, SPD-OReilly publication**
3. **The art of R programming by Norman Matloff, No Starch Press, San Francisco.**